中山大学计算机学院

人工智能实验报告

课程名称：Artificial Intelligence

教学班级	超算班	专业	信息与计算科学
学号	21307261	姓名	王健阳

一，实验题目

知识图谱推理

二，实验内容

算法原理

根据知识图谱中所有不含有目标谓词的知识，与变量排列组合，以表格的形式展开，分别计算其在各个情况下的正反例以及相应的增益值，然后取出增益值最大的那一项对应的规则，与下一次循环构造的每一项规则最与运算，直到出现一种只具有正例而不具有反例的情况，即找到了目标知识的推理过程。

而计算正反例，则是将每一项的规则带入到知识图谱中，看存在哪些变量替换，再依次将这些替换带到目标知识中去，看代换之后是否可以在图谱上找到与代换之后目标知识的变量相匹配的实例，再根据是否属于目标谓词来判断是正例还是反例

伪代码

```
graph = Initial()
# 在整个图谱上搜索正反例个数
pos, neg = CreateExample(graph, goal_knowledge)
var rules_append = [], table = [], changes = []

# 一次while循环即为目标知识推导出一个前提知识
while not Finished(table) begin
    table.clear()
    # 按照每一行来依次构建表格，并在表格项构造函数中就计算好各项数值
    for predicate in predicate_set begin
        for xy in Variable_list begin
            rules = [predicate + xy]
            rules.extend(rules_append)
             temp = TableItem(changeToEdge(goal_knowledge), rules, graph)
             table.append(temp)
        end
    end
    # 找出表格里面的最大增益值，并加入到新规则列表中
    var index = -1, max_value = -1
    for item in table begin
        if item.value > max_value begin
```

```
                max_value, index = item.value, table.index(item)
            end
        end
        rules_append = table[index].rules
    end
    var inferences = [], conclusion = []
    # 一些规范化输出代码
    for rule in rules_append begin
        conclusion.append(changeToStr(rule))
    end
    print('(' + ' ^ '.join(conclusion) + ') -> ' + goal_knowledge)
```

关键代码展示

> 主函数代码

```
# 根据测试用例初始化知识图谱，直接运行即可(但是需要修改functions.py里面的goal_knowledge)
graph = Initial()

# 在整个图谱上搜索正反例个数
pos, neg = CreateExample(graph, changeToEdge(goal_knowledge))
rules_append, table, changes = [], [], []

# 一次while循环即为目标知识推导出一个前提知识
while not IsFinished(table):
    table.clear()
    # 按照每一行来依次构建表格，并在表格项构造函数中就计算好各项数值
    for predicate in predicate_set:
        for xy in Variable_list:
            rules = [changeToEdge(predicate + xy)]
            rules.extend(rules_append)
            temp = TableItem(changeToEdge(goal_knowledge), rules, graph)
            # 计算每一项的增益值
            temp.FOIL_Gain(pos, neg)
            table.append(temp)
    # printTable(table)

    # 找出表格里面的最大增益值，并加入到新规则列表中
    index, max_value = -1, -1.0
    for item in table:
        if item.value != 'NA' and float(item.value) > max_value:
            max_value, index = float(item.value), table.index(item)
    rules_append = table[index].rules
    # 记录对应的变量替换，便于后期查找所有符合目标谓词的知识组合
    changes = table[index].substitution
    # 更新全局的正反例数值
    pos, neg = table[index].pos, table[index].neg
    for rule in rules_append:
        # rule.Print()
        pass
```

```
    # print('----------------------while ended----------------------------')


inferences, conclusion = [], []
# 一些规范化输出代码
for rule in rules_append:
    conclusion.append(changeToStr(rule))
print('(' + ' ^ '.join(conclusion) + ') -> ' + goal_knowledge)
# print(changes)

for change in changes:
    mark = 1
    tt = changeToEdge(goal_knowledge)
    if change:
        tt.st, tt.ed = change[tt.st], change[tt.ed]
        # 每一个替换都对应着一个实例，剔除掉原先就属于知识图谱的实例之后（例如本来就存在的Father(David, Mike)），再依次输出
        for edge in graph.edge_set:
            if changeToStr(tt) == changeToStr(edge):
                mark -= 1
        if mark:
            inferences.append(changeToStr(tt))
for inference in inferences:
    print(inference)


file = open("../result/result.txt", 'w')
file.write(str(len(graph.edge_set)) + '\n')
for edge in graph.edge_set:
    file.write(changeToStr(edge) + '\n')
file.write('\n\n(' + ' ^ '.join(conclusion) + ') -> ' + goal_knowledge + '\n')
for inference in inferences:
    file.write(inference + '\n')
file.close()
```

> 类代码

```
class Edge:
    def __init__(self, pre, st, ed):
        self.predicate = pre
        self.st = st
        self.ed = ed

    # 判断两条边的顶点值是否匹配（都是不同的常量就要返回false）
    def ValueMatch(self, goal_after_change):
        if IsVariable(goal_after_change.st) and self.ed == goal_after_change.ed:
            return True
        elif IsVariable(goal_after_change.ed) and self.st == goal_after_change.st:
            return True
        elif self.st == goal_after_change.st and self.ed == goal_after_change.ed:
            return True
        elif IsVariable(goal_after_change.st) and
```

```
IsVariable(goal_after_change.ed):
            return True
        else:
            return False

    # 打印函数，为打印类对象提供便利
    def Print(self):
        print(self.predicate + '(' + self.st + ', ' + self.ed + ')')


# 知识图谱类
class Graph:
    def __init__(self):
        self.edge_set = set()

    # 参数为规则，返回知识图谱中符合该规则的所有变量替换（是一个列表）
    # 此处简化了讨论，仅限于规则列表长度不大于二的情形
    def f1(self, rules: list):
        substitution = []
        # 只含有单个规则束缚
        if len(rules) == 1:
            rule = rules[0]
            for edge in self.edge_set:
                if edge.predicate == rule.predicate:
                    temp = {}
                    temp[rule.st] = edge.st
                    temp[rule.ed] = edge.ed
                    substitution.append(temp)
        # 含有两个规则束缚
        else:
            state = PredicateMatch(rules[0], rules[1])
            for e1 in self.edge_set:
                for e2 in self.edge_set:
                    if [rules[0].predicate, rules[1].predicate] == [e1.predicate,
e2.predicate]:
                        temp = {}
                        if state == 'same' and [e1.st, e1.ed] == [e2.st, e2.ed]:
                            temp[rules[0].st], temp[rules[0].ed] = e1.st, e1.ed
                        elif state == 'reverse' and [e1.st, e1.ed] == [e2.ed,
e2.st]:
                            temp[rules[0].st], temp[rules[0].ed] = e1.ed, e1.st
                        elif state == '00' and e1.st == e2.st:
                            temp[rules[0].st], temp[rules[0].ed],
temp[rules[1].ed] = e1.st, e1.ed, e2.ed
                        elif state == '01' and e1.st == e2.ed:
                            temp[rules[0].st], temp[rules[0].ed],
temp[rules[1].st] = e1.st, e1.ed, e2.st
                        elif state == '10' and e1.ed == e2.st:
                            temp[rules[0].st], temp[rules[0].ed],
temp[rules[1].ed] = e1.st, e1.ed, e2.ed
                        elif state == '11' and e1.ed == e2.ed:
                            temp[rules[0].st], temp[rules[0].ed],
temp[rules[1].st] = e1.st, e1.ed, e2.st
                        else:
```

```
                            pass
                        substitution.append(temp)
        return substitution


# 表格项类，代表了表格里面的每一行
class TableItem:
    def __init__(self, goal: Edge, rules: list, graph: Graph):
        self.graph = graph
        self.goal = goal
        self.pos, self.neg = 0, 0
        self.rules = rules
        self.value = 0
        # 对于表格的每一行的规则，通过调用graph的函数来计算对应的变量替换
        self.substitution = graph.f1(self.rules)
        for rule in rules:
            #print(changeToStr(rule), end=' ^ ')
            pass
        #print('\b\b      ', self.substitution)
        # 如果列表非空，则开始进行正反例的计算
        if self.substitution:
            self.CalcExample()

    # 根据替换对目标知识进行实例化
    def Instantiation(self):
        temp_list = []
        for item in self.substitution:
            if item:
                temp = Edge(self.goal.predicate, self.goal.st, self.goal.ed)

                for key in item.keys():
                    temp.st = item[key] if key == self.goal.st else temp.st
                    temp.ed = item[key] if key == self.goal.ed else temp.ed
                #temp.Print()
                # 此处转换为字符串是为了用set()进行去重
                temp_list.append(changeToStr(temp))
        return list(set(temp_list))

    def CalcExample(self):
        # 先依照替换，对目标知识进行变量替换（不改变其原始值）
        goals_after_change = self.Instantiation()
        #print('goals_after_change: ', goals_after_change, end='\n\n')
        # 对于目标知识每一种可能的实例化，对图谱的边集合进行遍历，计算符合条件的正反例数
        for goal_after_change in goals_after_change:
            #print(goal_after_change)
            goal_after_change = changeToEdge(goal_after_change)
            for edge in self.graph.edge_set:
                # 被计算进去的前提是，变量不能有冲突（例如对应位置有不同的常量）
                if edge.ValueMatch(goal_after_change):
                    # 若变量匹配但谓词不同，就是反例，反之就是正例
                    if edge.predicate == goal_after_change.predicate:
                        self.pos += 1
                    else:
                        self.neg += 1
```

```
                else:
                    continue

    # 表格项的便捷输出函数
    def Print(self):
        rules = []
        for rule in self.rules:
            rules.append(changeToStr(rule))
        rules_str = ' ^ '.join(rules)
        print(rules_str, '    ^m+ =', self.pos, '   ^m- =', self.neg, '    Gain:',
self.value)

    # 计算增益值的函数
    def FOIL_Gain(self, pos, neg):
        if self.pos == 0:
            self.value = 'NA'
        else:
            self.value = str(round(self.pos * (math.log2(self.pos / (self.pos +
self.neg)) - math.log2(pos / (pos + neg))), 2))

    # 状态函数，判断该项是否已经彻底推导出结论
    def State(self):
        return self.pos and not self.neg
```

> 函数部分代码

```
# 判断是否是变量的基本函数
def IsVariable(ss: str):
    return ss in ['x', 'y', 'z']

# 判断是否已经得出正确的推导过程
def IsFinished(lst: list):
    for item in lst:
        if item.State():
            return True
    return False

# 进行边匹配（返回两个端点之间的关系，是完全相等，相反还是只有一个相等，并返回匹配索引）
def PredicateMatch(e1: Edge, e2: Edge):
    if [e1.st, e1.ed] == [e2.st, e2.ed]:
        return 'same'
    elif [e1.st, e1.ed] == [e2.ed, e2.st]:
        return 'reverse'
    else:
        if e1.st == e2.st:
            return '00'
        elif e1.st == e2.ed:
            return '01'
        elif e1.ed == e2.st:
            return '10'
        elif e1.ed == e2.ed:
            return '11'
```

```
        else:
            return 'false'


# Edge类和str的互化
def changeToEdge(pre: str):
    predicate = pre[:pre.index('(')]
    elements = pre[pre.index('(') + 1: -1].split(', ')
    return Edge(predicate, elements[0], elements[1])


def changeToStr(pre: Edge):
    return pre.predicate + '(' + pre.st + ', ' + pre.ed + ')'


# 初始化函数（含参不需输入）
def DataProc(test: list):
    mygraph = Graph()
    for knowledge in test:
        predicate = knowledge[:knowledge.index('(')]
        [st, ed] = knowledge[knowledge.index('(') + 1: -1].split(', ')
        predicate_set.add(predicate)
        mygraph.edge_set.add(Edge(predicate, st, ed))
    predicate_set.discard(goal_knowledge[:goal_knowledge.index('(')])

    print(len(test))
    for item in test:
        print(item)
    print()
    print(goal_knowledge, end='\n--------------------------------\n')
    return mygraph

# 初始化函数（不含参需要输入）
def Initial():
    global goal_knowledge
    mygraph = Graph()
    num = int(input())
    for ii in range(num):
        knowledge = changeToEdge(input())
        mygraph.edge_set.add(knowledge)
        predicate_set.add(knowledge.predicate)
    goal_knowledge = input()
    predicate_set.discard(goal_knowledge[:goal_knowledge.index('(')])
    return mygraph


# 没任何前提得条件下计算正反例个数
def CreateExample(graph: Graph, goal: Edge):
    pos, neg = 0, 0
    for edge in graph.edge_set:
        if goal.predicate == edge.predicate:
            pos += 1
        else:
```

```
                neg += 1
        return pos, neg
```

创新点 & 优化

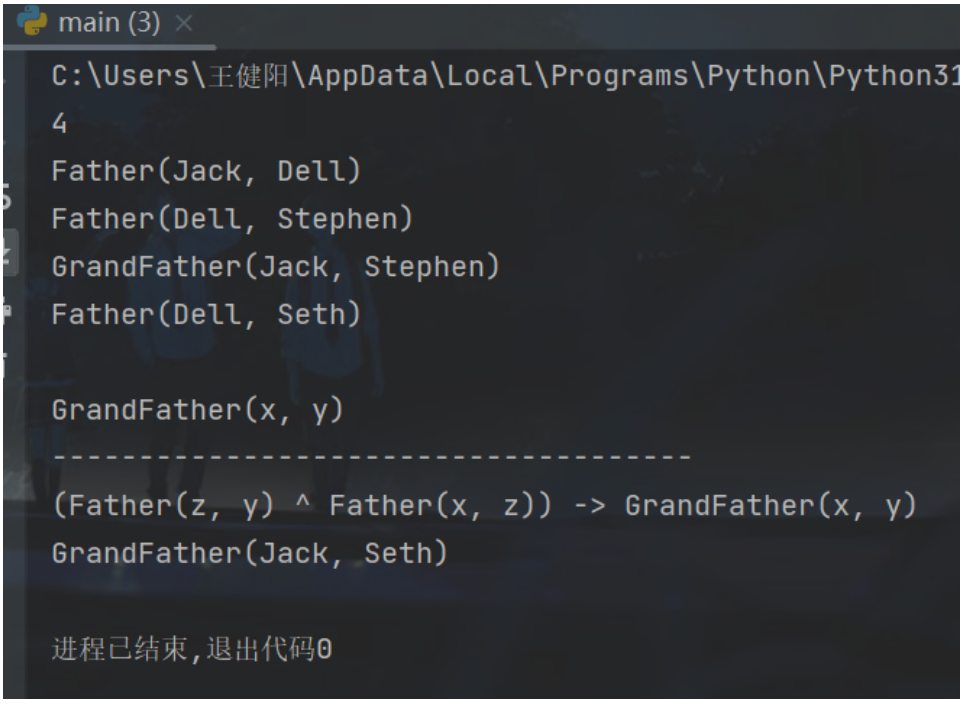
时间较赶，暂时没想出好的优化））

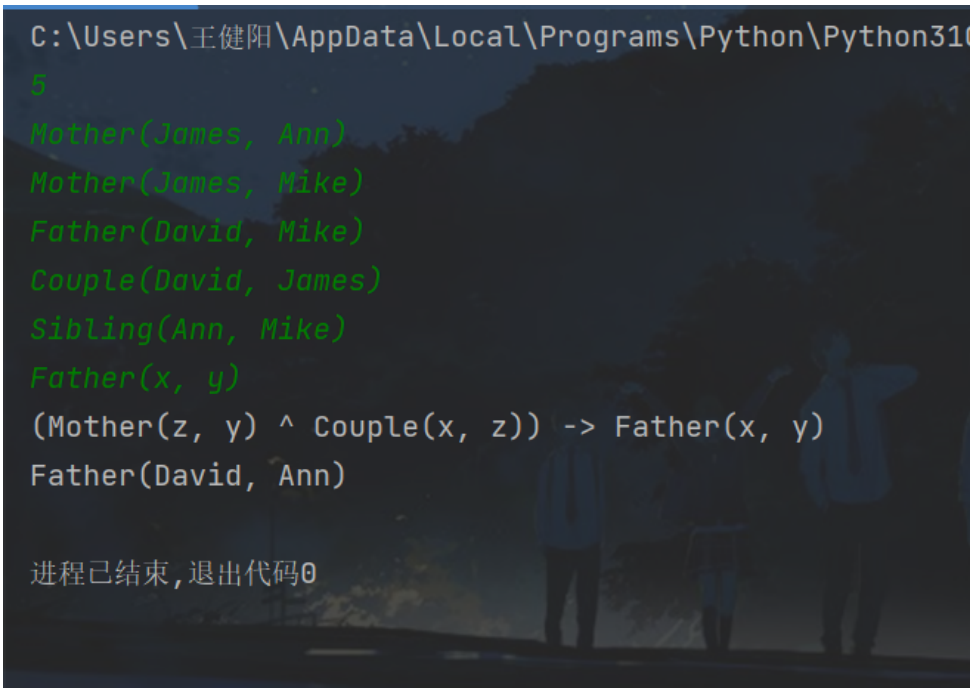
三，实验结果及分析

算法结果展示实例

pdf用例以及实验用例的输出结果如下：

（均成功完成了推理任务）





四，思考题

暂无思考题

```

```

五，参考资料

1.实验python基础pdf

2.CSDN